iRobot Education's Create 3

iRobot ROS2 Playground

Explore the following code snippets used to program the Create 3.

Commands

robot.move(16) robot.move(16, speed=20, useGoal=False)	Command the robot to move forward a specified number of centimeters. Example #1 shows 16 cm distance. Example #2 shows a changeable speed 20 cm/s and the option to disable goals
robot.turn_left(90) robot.turn_left(90, speed=20, useGoal=False) robot.turn_right(90) robot.turn_right(90, speed=20, useGoal=False)	Command the robot to rotate clockwise (right) or counter-clockwise (left) for a specified number of degrees. Example #1 shows robot driving counter-clockwise 90°. Example #2 shows a changeable speed 20 cm/s and the option to disable goals
robot.arc_left(90, 4) robot.arc_left(90, 4, speed=20, useGoal=False) robot.arc_right(90, 4) robot.arc_right(90, 4, speed=20, useGoal=False)	Command the robot to drive forward or backward in an arc, either clockwise (right) or counter-clockwise (left) for a specified number of degrees along a curve of a specified radius. Example #1 shows robot driving counter-clockwise 90° around a curve with a 4 cm radius. Example #2 shows a changeable speed 20 cm/s and the option to disable goals
robot.reset_navigation()	Command resets the robot's navigation grid origin of (0, 0, 90º) with the center of the robot at the origin whose X axis goes through the right wheel and Y axis goes through the bumper.
robot.navigate_to(16, 16) robot.navigate_to(0, 0, 45) robot.navigate_to(0, 0, 45, speed=20, useGoal=False)	Command the robot to navigate across an invisible grid of centimeter increments. The robot's position is reset to (0, 0, 90º) when the program is started. Example #1 shows a robot navigating to (16 cm, 16 cm) without setting a final heading. Example #2 shows a robot navigating to (0 cm, 0 cm) with a final heading of 45 degrees. Example #3 same as #2 and shows a changeable speed 20 cm/s and the option to disable goals
robot.play_note(440, 0.25) robot.play_note(Note.A4, 0.25)	Command the robot to play a musical note of a specified frequency for a specified duration. Example #1 and #2 play the note A4 for .25 seconds.
robot.dock() robot.undock()	Triggers a docking action and undocking action it will wait till it completes.

Setters

robot.set_lights_on_rgb(0, 0, 255) robot.set_lights_spin_rgb(0, 0, 255) robot.set_lights_blink_rgb(0, 0, 255) robot.set_lights_off()	Set the robot's lights to show various colors and patterns. Examples #1-3 set the robot's lights blue, using the fully on, spinning, and blinking animations, respectively. Example #4 turns the robot's lights off.
robot.set_wheel_speeds(10, -10)	Set the robot's wheel speed (in cm/s) and direction on either side (left/right). Example shows the robot's left wheel driving forward 10 cm/s and right wheel driving backward 10 cm/s. The robot will rotate in place, clockwise.
robot.set_left_speed(5) robot.set_right_speed(-5)	Sets one of the robot's wheel motor speeds (in cm/s); the other will continue unchanged.
robot.set_lights(leds)	Set lights based on LEDs list.
robot.print('Robot initiated') robot.warning('hit wall') robot.error('crashed into wall')	Print statements using: node.logger().info() node.logger().warning() node.logger().error()

Getters

robot.get_battery_level()	Gets the robot's battery level. Returns a float of percentage 0 to 100.
robot.get_position()	Gets the robot's current internal position. Returns an object with X, Y and angle.
robot.get_accelerometer()	Gets the current values reported by the robot's accelerometer. Returns an object with X, Y and Z.
robot.get_bumpers()	Gets the current state of the bumper sensors. Returns an object with Booleans of each bumper position. right, front_right, front_center, front_left and left. Multiple can be true at once.
robot.get_touch_sensors()	Gets the current values reported by the robot's touch sensors. Returns an object with Booleans of each button. button_1, button_power and button_2. Multiple can be true at once.
robot.get_ir_proximity()	Gets the values of the robot's IR proximity sensors. Returns a array list of sensor values with the left most value being the left most sensor.
robot.get_cliff_sensors()	Gets Boolean values of the robot's cliff sensors. Returns an object with Booleans of each sensor. side_right, front_right, front_left and side_left. Multiple can be true at once
robot.get_docking_values()	Get Docking values and which sensor is triggered. Returns a object with is_docked, dock_visible. It also returns what sensor is triggered either 0 or 1 and what range it is in. Either greenBuoy, redBuoy and forcefield multiple of these can be true at once.

Tools

robot.tools.convertToEuler(0, 0, 0.5, 0.5)	Converts Quaternion Rotations into Euler Angles. Euler angles are in roll, pitch and yaw. This is useful to understand the complex quaternion from the robots' orientations. The only value that is relevant to this bot is the yaw (also referenced as z)
robot.tools.convertToQuaternion(0, 0, 0.5)	Converts Euler Angles into Quaternion Rotations. Quaternions are used mostly in the orientation of this robot. Can be found when calling odom, imu, and navigateTo. By placing roll and pitch as 0 and yaw as the desired angle in radians. It will return the correct orientation for the bot.
robot.tools.led.adjustBrightness(led, 50)	This will automatically set the LED object 'LedColor' to a desired brightness level.
robot.tools.led.adjustRotationBrightness(led, 0.5, 120) robot.tools.led.adjustRotationBrightness(led, 0.5, 120, 90)	This will automatically set the LED object 'LedColor' to a desired brightness level based on a directional rotation (0 to 1) and angle location of LED. This will create a fade effect while transitioning to another LED. Example #1 'led' is set color object, 0.5 is the rotation around a circle out of 1. And 120 is the LED position on that circle. Example #2 is the exact same except the span is set. This setting adjusts the total angle that there should be light emitted from the circle any LED's in this range will be lit up.
robot.tools.led.getHuePecentage(0.47) robot.tools.led.getHuePecentage(0.47, 0, 270)	This will return a Color Hue based on percentage (from 0 to 1) or just pass it a Hue value. Example #1 returns a LED Color object with hue percentage of 0.47 Example #2 Is the same but will return a hue between 0 and 270 out of 360 meaning that only the colors within that range will be returned.
robot.tools.lidar.getRotation(sensors)	By passing the array list of IR sensors value it will return a float of the degree angle, based on the highest value in the list.
robot.toolslir.getMotionLightring(sensors) robot.tools.ir.getMotionLightring(sensors, 0, 255, 0)	This will return a list of 'LedColor' objects that reflect the location of the highest sensor value. Example #1 will return list of LEDs that move around the lightring based on a hue that is found from the sensors value. Example #2 same as before but allows to set specific color as override as RGB.

Compatible with the Create® 3 Robot

RGB Color Reference Sheet

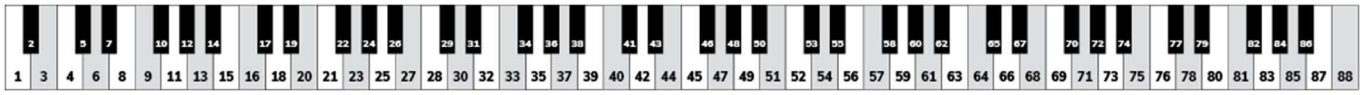
Use these RGB values when setting the color of the LED's in the Ring Light.

100% Brightness	50% Brightness
(255, 0, 0)	(127, 0, 0)
(255, 64, 0)	(127, 32, 0)
(255, 115, 0)	(127, 57, 0)
(0, 255, 0)	(0, 127, 0)
(101, 197, 181)	(50, 98, 90)
(0, 0, 255)	(0, 0, 127)
(111, 71, 127)	(55, 35, 63)
(255, 255, 255)	(127, 127, 127)
(60, 60, 60)	(0, 0, 0)

Compatible with the Create® 3 Robot

Note Frequencies for Python

The following note values are for use in the iRobot ROS2 package



Key #	Note	Hz	Constant
1	A0	**27.50**	
2	A♯0/B♭0	29.14	
3	B0	30.87	
4	C1	32	C1
5	C♯1/D♭1	34	C1_SHARP
6	D1	36	D1
7	D♯1/E♭1	38	D1_SHARP
8	E1	41	E1
9	F1	43	F1
10	F♯1/G♭1	46	F1_SHARP
11	G1	48	G1
12	G♯1/A♭1	51	G1_SHARP
13	A1	**55**	**A1**
14	A♯1/B♭1	58	A1_SHARP
15	B1	61	B1
16	C2	65	C2
17	C♯2/D♭2	69	C2_SHARP
18	D2	73	D2
19	D♯2/E♭2	77	D2_SHARP
20	E2	82	E2
21	F2	87	F2
22	F♯2/G♭2	92	F2_SHARP
23	G2	97	G2
24	G♯2/A♭2	103	G2_SHARP
25	A2	**110**	**A2**
26	A♯2/B♭2	116	A2_SHARP
27	B2	123	B2
28	C3	130	C3
29	C♯3/D♭3	138	C3_SHARP
30	D3	146	D3

Key #	Note	Hz	Constant
31	D♯3/E♭3	155	D3_SHARP
32	E3	164	E3
33	F3	174	F3
34	F♯3/G♭3	184	F3_SHARP
35	G3	195	G3
36	G♯3/A♭3	207	G3_SHARP
37	A3	**220**	**A3**
38	A♯3/B♭3	233	A3_SHARP
39	B3	246	B3
40	C4	261	C4
41	C♯4/D♭4	277	C4_SHARP
42	D4	293	D4
43	D♯4/E♭4	311	D4_SHARP
44	E4	329	E4
45	F4	349	F4
46	F♯4/G♭4	369	F4_SHARP
47	G4	391	G4
48	G♯4/A♭4	415	G4_SHARP
49	A4	**440**	**A4**
50	A♯4/B♭4	466	A4_SHARP
51	B4	493	B4
52	C5	523	C5
53	C♯5/D♭5	554	C5_SHARP
54	D5	587	D5
55	D♯5/E♭5	622	D5_SHARP
56	E5	659	E5
57	F5	698	F5
58	F♯5/G♭5	739	F5_SHARP
59	G5	783	G5
60	G♯5/A♭5	830	G5_SHARP

Key #	Note	Hz	Constant
61	A5	**880**	**A5**
62	A♯5/B♭5	932	A5_SHARP
63	B5	987	B5
64	C6	1046	C6
65	C♯6/D♭6	1108	C6_SHARP
66	D6	1174	D6
67	D♯6/E♭6	1244	D6_SHARP
68	E6	1318	E6
69	F6	1396	F6
70	F♯6/G♭6	1479	F6_SHARP
71	G6	1567	G6
72	G♯6/A♭6	1661	G6_SHARP
73	A6	**1760**	**A6**
74	A♯6/B♭6	1864	A6_SHARP
75	B6	1975	B6
76	C7	2093	C7
77	C♯7/D♭7	2217	C7_SHARP
78	D7	2349	D7
79	D♯7/E♭7	2489	D7_SHARP
80	E7	2637	E7
81	F7	2793	F7
82	F♯7/G♭7	2959	F7_SHARP
83	G7	3135	G7
84	G♯7/A♭7	3322	G7_SHARP
85	A7	**3520**	**A7**
86	A♯7/B♭7	3729	A7_SHARP
87	B7	3951	B7
88	C8	4186	C8